

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to distinguish the purpose and scope of Static Application Security Testing (SAST) from Dynamic Application Security Testing (DAST) and Interactive Application Security Testing (IAST).
- By the end of this lecture, you will be able to identify which classes of vulnerabilities each testing methodology is best equipped to detect.
- By the end of this lecture, you will be able to determine the appropriate phase of the software development lifecycle for deploying SAST, DAST, and IAST tooling.
- By the end of this lecture, you will be able to apply the lessons of the Equifax 2017 breach to justify continuous runtime testing against deployed applications.
- By the end of this lecture, you will be able to design a layered security testing strategy that combines all three methodologies to achieve comprehensive application coverage.

Introduction

Modern application security testing is built on a foundational insight: no single tool, applied at a single point in time, can see the full attack surface of a software system. Vulnerabilities exist in at least three distinct dimensions -- in the written code itself, in the behaviour of the running system, and in the space where code executes under realistic conditions. Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), and Interactive Application Security Testing (IAST) each address one of those dimensions. Using them in combination produces a defence that is qualitatively different from any one approach applied in isolation.

The CCSP examination tests candidates not only on what each methodology is but on when each is used, why it is used at that stage, and what categories of vulnerability it is uniquely positioned to find. The Equifax breach of 2017 -- which exposed the personal and financial data of approximately 147 million people -- is a widely referenced case study precisely because it demonstrates what happens when DAST coverage is absent. Attackers exploited a known vulnerability in Apache Struts that a dynamic test against the deployed application would have detected, regardless of what the source code showed or what patches the security team believed had been applied.

As a Cloud Security Practitioner, you will frequently be asked to advise on how security testing fits into a CI/CD pipeline, what tooling to require in a development programme, and how to justify the cost of multiple testing modalities. The framework introduced in this lecture gives you the conceptual vocabulary and the practical rationale to do that convincingly.

Core Concepts Explained

Static Application Security Testing (SAST)

Definition: SAST analyses source code, bytecode, or binary code without executing the application, identifying security-relevant patterns such as SQL injection sinks, hardcoded credentials, insecure cryptographic usage, and unhandled exceptions.

Business relevance: Finding vulnerabilities in code before it is compiled or deployed is the cheapest possible remediation point. A flaw caught in the editor costs a developer thirty minutes to fix; the same flaw found in production after a breach costs millions in incident response, regulatory fines, and reputational damage.

Cloud Security Practitioner relevance: Cloud Security Practitioners evaluate SAST integration as part of DevSecOps programme design. A key best practice is embedding SAST into the IDE and CI pipeline so developers receive immediate feedback during the act of writing code, normalising security as a development activity rather than a gate applied at the end.

Real-world example: A financial services firm building a loan-origination microservice integrates a SAST tool into its GitHub Actions CI pipeline. When a developer submits a pull request, the tool traces data flows through the new code and flags that a variable derived from a query parameter reaches a dynamic SQL string without sanitisation three function calls later. The finding is surfaced as a pull-request comment before the code is merged, reviewed, or deployed. The developer fixes it in the same working session. Because the SAST tool follows data flows across module boundaries rather than checking only the function where the parameter first appears, it catches the flaw even though the vulnerable sink is in a shared utility library that the developer did not author.

Risks or limitations: SAST produces false positives, which cause developer fatigue if thresholds are not tuned and findings are not triaged. It cannot detect runtime-only behaviours such as misconfigured web server headers or session tokens that fail to expire. The best practice to counter false-positive fatigue is to configure the tool to break builds only on high-confidence, high-severity findings, routing lower-confidence findings to a separate review queue rather than blocking development flow.

Dynamic Application Security Testing (DAST)

Definition: DAST tests the running application by sending crafted HTTP requests and observing responses, simulating external attacker behaviour to find vulnerabilities that only emerge at runtime, such as verbose error pages, session management flaws, and misconfigured cross-origin policies.

Business relevance: DAST provides an attacker's-eye view of the deployed application. Because it interacts through external interfaces, it finds exactly the categories of weakness that external threat actors probe. In the Equifax case, a DAST scan against the production system would have revealed that an unpatched Apache Struts version was actively running, regardless of what the internal patch documentation stated.

Cloud Security Practitioner relevance: Cloud Security Practitioners specify DAST as a required gate in pre-production pipelines and as a scheduled control in production. The best practice is to run DAST against a staging environment that mirrors production as closely as possible, ensuring that the runtime configuration tested is representative of what attackers will encounter.

Real-world example: A healthcare SaaS provider runs automated DAST scans against its patient-portal staging environment as part of every weekly release cycle. During one scan, the tool submits a GET request to the password-reset endpoint with a predictable, sequential reset token and receives a valid response -- revealing that the token generation function lacks sufficient entropy. The development team had reviewed the reset logic in code review and judged it acceptable because the function appeared correct in isolation. The DAST tool

found the weakness because it interacted with the live token-generation behaviour across hundreds of automated requests, accumulating the statistical evidence that no code review can produce. The same scan also flagged that the application server was returning an X-Powered-By header revealing the framework version, a configuration exposure invisible in source code.

Risks or limitations: DAST requires a running environment, so it cannot be applied during early development. Aggressive scanning can disrupt test environments if rate limiting is not configured appropriately. In cloud-native architectures, ephemeral environments must be provisioned specifically for DAST execution, which requires pipeline automation. The best practice is to provision a dedicated, production-mirrored staging environment that DAST can probe without risk to real users or data.

Interactive Application Security Testing (IAST)

Definition: IAST deploys lightweight agents inside the running application that instrument it during active test execution, observing internal data flows and security-relevant operations in real time as functional or automated test suites exercise the code.

Business relevance: IAST bridges the gap between SAST and DAST, catching flaws that exist in the code but only become visible under realistic runtime conditions. It produces fewer false positives than SAST because findings are confirmed by actual execution paths, and it sees more internal detail than DAST because it instruments from within the process.

Cloud Security Practitioner relevance: Cloud Security Practitioners evaluate IAST as a complement to SAST in the CI pipeline, particularly in organisations that already maintain comprehensive automated functional test suites. The best practice is to run IAST agents during existing regression test cycles, so that coverage increases without requiring a separate testing effort.

Real-world example: A logistics platform has an extensive Selenium-based functional test suite that exercises its shipment-tracking application as part of every CI build. The team adds an IAST agent to the test runner. During the next regression cycle, the agent observes a data flow in which a shipment identifier supplied in a URL parameter is passed to a JDBC prepared statement but bypasses a secondary validation layer in a feature-flag-controlled code path. The flaw was invisible to SAST because the agent in the IAST tool saw the actual execution context, including the feature flag state. It was invisible to prior DAST scans because the code path is only reachable for authenticated carrier accounts with a specific role. The IAST agent found it because it observed internal execution during the functional tests that already exercised that role.

Risks or limitations: IAST requires instrumentation of the application, which introduces a performance overhead that must be acceptable in the test environment. Coverage is bounded by the coverage of the underlying test suite: if the tests do not exercise a code path, IAST cannot see it. The best practice is to use IAST alongside a high-coverage automated test suite and to review IAST coverage reports to identify which code paths remain uninstrumented.

The Testing Lifecycle: Matching Each Methodology to Its Optimal Phase

Definition: The effectiveness of each testing modality is tightly coupled to where in the software development lifecycle it is applied. SAST belongs earliest, integrated into development tooling; DAST belongs in pre-production; IAST operates during active test cycles in a staged environment.

Business relevance: The cost of finding and fixing a vulnerability rises sharply with the distance between the moment it is introduced and the moment it is discovered. Matching each tool to the lifecycle phase where it is most effective is not merely an efficiency consideration -- it directly determines which vulnerability classes are caught before production and which are not.

Cloud Security Practitioner relevance: Cloud Security Practitioners design DevSecOps pipelines by mapping each testing modality to a specific pipeline gate. The best practice is to configure SAST as a mandatory pull-request check, DAST as a pre-production release gate, and IAST as a standard component of regression test execution, with each gate having a defined owner, defined pass/fail criteria, and a defined escalation path.

Real-world example: A cloud-native payments company documents its security testing pipeline explicitly. SAST runs in the IDE via a plugin and again as a pull-request check in GitHub Actions; high-severity findings block merge. A staging environment that mirrors production -- including its TLS configuration, web server version, and API gateway rules -- is provisioned automatically for each release candidate. DAST runs against that environment using OWASP ZAP in a dedicated pipeline stage; the release is not promoted to production unless DAST exits with zero high or critical findings. IAST agents run during the nightly regression suite; findings are routed to the development team for triage. The programme produces three independently sourced vulnerability reports per release cycle, each addressing a different face of the application attack surface.

Risks or limitations: The most common programme failure is treating the tools as redundant and selecting only one while omitting the others. The Equifax breach illustrates the consequence: the source code was available, the vulnerability was known and patched in the official repository, but the deployed application was never tested dynamically. The best practice is to treat SAST, DAST, and IAST as complementary controls, each assigned to its appropriate lifecycle phase with a clear owner responsible for acting on its findings.

Practical Applications

The following scenarios show how security professionals apply SAST, DAST, and IAST in real programme design decisions -- not as abstract tools but as controls with specific owners, pipeline positions, and failure modes.

Integrating SAST into a DevSecOps Pipeline Without Blocking Delivery

A Cloud Security Practitioner at a software-as-a-service company is asked to add SAST to the CI pipeline without slowing release velocity. The practitioner configures the SAST tool with a tiered severity policy: Critical or High severity findings that match high-confidence rules break the pull-request check and prevent merge until resolved. Medium and Low findings are reported in a dashboard reviewed weekly by the development team but do not block delivery. The practitioner also defines a suppression workflow requiring a documented justification and a reviewer approval, creating an audit trail without allowing unreviewed suppressions. Over the first quarter, the false-positive rate drops as the rule set is tuned to the organisation's technology stack, and developer acceptance improves because the tool surfaces findings that developers recognise as real. The key design decision: SAST rules are tuned for code-level data-flow issues rather than duplicating what runtime scanning already covers, keeping each tool focused on its comparative advantage.

Using DAST to Surface Configuration Exposures Missed by Code Review

A development team believes its API has been hardened by thorough code review and SAST. A Cloud Security Practitioner schedules a DAST scan against the staging environment before the next production release. The scan reveals three findings code review missed entirely: the application error handler returns a full Java stack trace for unexpected inputs, leaking the internal framework version; the API accepts CORS preflight requests from any origin without credential validation; and a legacy endpoint that was supposed to be disabled returns a 200 response with partial data. None are source code vulnerabilities. All three are runtime configuration and integration issues that only appear when the running system is tested against realistic HTTP traffic. The practitioner flags all three as pre-production blockers. The release is held until each is remediated and re-scanned to confirm resolution. The exercise reinforces the programme policy: DAST is not optional even when SAST passes, because the two tools examine fundamentally different things.

Deploying IAST During Regression Testing to Maximise Coverage Without Extra Effort

A healthcare information technology organisation maintains a large Selenium functional test suite that achieves 80 percent code coverage against its patient-record application. A Cloud Security Practitioner proposes deploying an IAST agent during the nightly regression run, arguing that the organisation already exercises most code paths and the IAST agent simply observes the security-relevant behaviour that occurs during those exercises. The proposal is approved with the condition that IAST agent overhead not exceed 15 percent of total test runtime. The agent is deployed, overhead is measured at 8 percent, and within three regression cycles it surfaces two findings: a deserialization call in a background-task processor that accepts an attacker-controlled input type, and a direct database query in an administrative function that bypasses the standard parameterization layer. Both required internal runtime observation to detect and both were remediated before the next scheduled release. The programme's conclusion: IAST provides a material return on an investment that is primarily the cost of instrumentation, because the test infrastructure it rides on already exists.

Best Practices

- **Embed SAST in both the developer's IDE and the CI pull-request gate.** Running SAST only in a scheduled nightly job means developers have moved on by the time findings are reported. IDE integration surfaces findings during the act of writing code, when the developer has full context and the fix cost is minimal. The CI gate provides a second enforcement point before code enters the shared codebase, ensuring that findings cannot be bypassed simply by skipping the IDE plugin.
- **Target DAST at a staging environment that mirrors production configuration, not just production code.** The most common DAST gap is running it against an environment that lacks production's web server configuration, TLS settings, and API gateway rules. Findings that depend on configuration -- verbose error pages, permissive CORS headers, exposed legacy endpoints -- will not appear in a simplified test environment. Invest in environment parity so DAST findings are valid representations of production risk.
- **Run IAST during existing high-coverage automated regression test cycles rather than scheduling separate IAST-specific sessions.** IAST coverage is bounded by the coverage of the test suite it instruments. If a high-coverage automated suite already exists, deploying IAST during that suite costs only the agent overhead and produces immediate returns. Creating a dedicated IAST test programme from scratch is unnecessary and delays adoption.
- **Treat SAST, DAST, and IAST findings as separately sourced vulnerability reports requiring independent triage.** Each methodology finds a different class of vulnerability. A DAST finding is not invalidated by a clean SAST result, and a SAST finding is not resolved by a passing DAST scan. Maintain separate finding queues with tool-appropriate triage criteria and separate remediation owners so that each finding type receives appropriate attention.
- **Validate the deployed state dynamically whenever a Critical or High CVE is disclosed for a component in your technology stack.** The Equifax breach occurred because the patch was known and documented internally, but nobody confirmed the patched version was running in the deployed production environment. DAST against the live system is the control that confirms deployment state. Establish a policy requiring a targeted DAST re-scan of any service affected by a Critical or High CVE within a defined SLA, independent of the scheduled scan cycle.

Common Mistakes and Misconceptions

Misconception: Treating SAST as sufficient and omitting DAST because the code has already been reviewed.

Correct approach: DAST finds runtime and configuration vulnerabilities that are invisible to code analysis. The best practice is to mandate DAST as a pre-production release gate with defined severity thresholds, treating it as a distinct and complementary control rather than a redundant one.

Misconception: Running DAST against a development environment that does not mirror the production configuration.

Correct approach: Configuration-dependent findings will not appear in a non-representative environment. The best practice is to provision a dedicated staging environment with production-equivalent configuration -- including web server settings, TLS certificates, and API gateway rules -- specifically for DAST execution.

Misconception: Assuming that SAST and DAST together are comprehensive and that IAST adds no additional value.

Correct approach: IAST finds vulnerabilities that exist in code but only surface under realistic execution conditions that neither static analysis nor external scanning can replicate. The best practice is to deploy IAST during high-coverage regression cycles to capture the class of findings that requires both code access and runtime confirmation.

Misconception: Allowing developers to silently suppress SAST findings without documented justification and reviewer approval.

Correct approach: Silent suppressions remove findings from visibility without accountability. The best practice is to require a named justification and a reviewer approval for every suppression, creating an audit trail that can be reviewed in the event of a future incident involving the suppressed code path.

table: SAST vs DAST vs IAST: Programme Design Reference

Use this comparison when advising on which tools to require at each pipeline stage or when evaluating a proposed DevSecOps testing programme.

Attribute	SAST	DAST	IAST
What it analyses	Source / bytecode / binary	Running application (external)	Running application (internal)
Execution required?	No	Yes	Yes
Primary lifecycle phase	Development / PR gate	Pre-production staging	Regression / test cycle
Finds code-level data flows	Yes -- strong	No	Yes -- strong
Finds runtime config exposures	No	Yes -- strong	Partial
Finds session / integration flaws	No	Yes	Partial
Typical false-positive	Higher (context-free)	Lower (runtime confirmed)	Lowest (execution confirmed)

rate			
Developer feedback speed	Fastest (IDE / PR)	Slower (requires env)	Moderate (test cycle)
Equifax gap addressed?	Partial (code only)	Directly (deployed state)	Partial (runtime paths)

How to use this: No single tool covers the full attack surface. A mature programme assigns each tool to its appropriate phase and treats findings as independently valid inputs requiring separate triage.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A development team runs SAST against every pull request and receives a clean result. The security team then runs DAST against the staging environment and finds that the application returns a detailed stack trace on unexpected inputs. Which of the following best explains this outcome?

- A) A. SAST missed the stack-trace issue because its rule set was not configured correctly.
- B) B. DAST found a runtime configuration behaviour that SAST cannot detect because SAST analyses code, not deployed system responses.
- C) C. The two tools are redundant and the DAST finding indicates a false positive.
- D) D. DAST is less reliable than SAST and its findings should be verified manually before acting on them.

Answer: B. DAST found a runtime configuration behaviour that SAST cannot detect because SAST analyses code, not deployed system responses.

Q2. The Equifax breach of 2017 is most directly cited as an example of what security testing failure?

- A) A. SAST was not integrated into the development pipeline, so the vulnerability was written into the code undetected.
- B) B. DAST was not run against the deployed application, so a known exploitable vulnerability in a running third-party component was not confirmed.
- C) C. IAST agents were not deployed, so the data flow to the vulnerable component was not observable at runtime.
- D) D. The organisation relied on a bug bounty programme instead of automated scanning.

Answer: B. DAST was not run against the deployed application, so a known exploitable vulnerability in a running third-party component was not confirmed.

Q3. An organisation wants to maximise IAST coverage without scheduling a separate IAST-specific test programme. What is the recommended best practice?

- A) A. Deploy IAST agents in the production environment so all real user traffic is instrumented.
- B) B. Run IAST during existing high-coverage automated regression test cycles so IAST coverage inherits the test suite's code coverage.
- C) C. Replace the automated regression suite with IAST-directed tests that specifically target security-sensitive code paths.
- D) D. Run IAST only against newly developed code, since existing code has already been validated by SAST.

Answer: B. Run IAST during existing high-coverage automated regression test cycles so IAST coverage inherits the test suite's code coverage.

Q4. Which vulnerability class is SAST best positioned to detect that DAST cannot?

- A) A. A web server header that reveals the application technology stack.
- B) B. A session token that remains valid after a user logs out.
- C) C. A variable derived from user input that reaches a SQL query sink three function calls later.
- D) D. A CORS policy that accepts preflight requests from any origin.

Answer: C. A variable derived from user input that reaches a SQL query sink three function calls later.

Q5. A Cloud Security Practitioner is designing a DevSecOps pipeline and must assign SAST, DAST, and IAST to specific stages. Which assignment correctly matches each tool to its most effective lifecycle phase?

- A) A. SAST in pre-production staging; DAST in the IDE during development; IAST in production monitoring.
- B) B. SAST at the pull-request gate during development; DAST in pre-production staging; IAST during automated regression testing.
- C) C. SAST and DAST both at the pull-request gate; IAST in pre-production staging.
- D) D. All three tools in the pre-production staging environment to consolidate findings into a single report.

Answer: B. SAST at the pull-request gate during development; DAST in pre-production staging; IAST during automated regression testing.

Reflection Questions

1. Consider a past or current application project in your organisation. At which phase of the development lifecycle are security vulnerabilities most commonly discovered? What would change if SAST findings were surfaced at the pull-request stage rather than after deployment?
2. The Equifax breach demonstrates that knowing a vulnerability exists in a component and confirming that the component is actually running in production are two different things. What controls exist in your current environment to close that gap?
3. IAST coverage depends on the quality of the test suite it instruments. How would you evaluate whether your organisation's current automated test coverage is sufficient to make IAST a worthwhile investment?
4. If asked to justify the cost of maintaining three separate security testing tools to a development team that argues SAST alone is sufficient, what three specific vulnerability classes would you cite to demonstrate that DAST and IAST find things SAST cannot?

Key Takeaways

- SAST analyses code without execution and is most effective when integrated into the developer's IDE and CI pull-request gate, where remediation cost is lowest.
- DAST tests the running application through its external interfaces, finding runtime and configuration vulnerabilities that are invisible in source code -- misconfigured headers, session management flaws, and exposed legacy endpoints.
- IAST deploys agents inside the running application and observes internal data flows during active test execution, combining code-level visibility with runtime realism.

- The Equifax breach of 2017 illustrates the consequence of a DAST gap: a known, patchable vulnerability in a deployed component went undetected because the live system was never tested dynamically, despite internal documentation about patching.
- SAST, DAST, and IAST are not redundant -- each finds a different class of vulnerability because each examines a different face of the application attack surface.
- A mature security testing programme assigns each tool to its appropriate lifecycle phase with separate finding queues, independent triage criteria, and named remediation owners.
- The cost of finding and fixing a vulnerability is inversely proportional to how early in the lifecycle it is found -- shifting testing left through SAST integration is the highest-leverage investment in most DevSecOps programmes.

Recommended Next Actions

Put this lecture to work within the next seven days:

5. Audit your current CI/CD pipeline and identify which of SAST, DAST, and IAST are integrated, at which stages, and what their pass/fail criteria are -- document any gaps and assign owners for each missing control.
6. For any Critical or High CVE disclosed against a component in your technology stack in the past six months, verify whether a DAST scan was run against the deployed application to confirm the component's runtime version, not only against the source repository.
7. Review your organisation's automated test suite coverage report and identify the three highest-risk code paths not currently exercised -- assess whether adding IAST coverage for those paths is feasible within the next sprint cycle.
8. Draft a one-page programme brief mapping SAST, DAST, and IAST to specific pipeline gates, assigning a named owner for each tool's findings and defining the escalation path for findings that block a release.